Computer Architecture
Lab 3: Superscalar RISC-V Processor

Institute of Computer Science and Engineering
National Yang Ming Chiao Tung University

revision: 10-4-2026

In this lab, you will design and implement a two-wide superscalar processor. We provide a functional datapath, and your task will be implementing the necessary control logic, instruction steering mechanism, and a scoreboard to enable the processor to execute two instructions per cycle efficiently.

This lab will provide you with experience in:

- The principles of superscalar processor design and multi-issue pipelines.
- How to implement control logic and instruction steering for in-order execution.
- The use of a scoreboard to manage dependencies and maximize instruction throughput.

1 Reminder

- Submission Deadline: 11/16 (Mon) 11:59 p.m.
- Please refer to Lab 0 for the academic integrity requirements.
- No sharing or distribution of lab materials is allowed.

2 Setup

2.1 Getting Started

Once you have the Lab 3 materials, extract them by entering the following commands:

```
% tar -xzf lab3.tar.gz
% cd lab3
% export LAB3_ROOT=$PWD
```

Within the lab root directory, you will find eight subdirectories, each serving a specific purpose:

- `build`: Contains the Makefile and compiled code.
- `imuldiv`: Integer multiply/divide unit.
- `riscvssc_part1`: Superscalar RISC-V processor source code.
- `riscvssc_part2`: Your optimized code.
- `tests`: Assembly test build system.
  - `riscv`: RISC-V assembly tests.
  - `scripts`: Miscellaneous scripts for build system.

- `ubmark`: Benchmarks for evaluation.
- `vc`: Contains additional Verilog components.

Most directories remain unchanged from the previous lab. The new additions are `riscvssc_part1` and `riscvssc_part2`. The `riscvssc_part1` directory contains the initial datapath for a two-wide superscalar processor, lacking only the scoreboard, steering logic, and control logic.

2.2 Building the Project

As usual, we'll start by compiling the reference processors and running the RISC-V assembly tests:

```
% cd $LAB3_ROOT/tests
% mkdir build && cd build
% ../configure --host=riscv32-unknown-elf
% make
% ../convert
% cd $LAB3_ROOT/build
% make check-asm-riscvssc_part1
% make check-asm-rand-riscvssc_part1
```

The methodology remains the same as in Lab 2. For details on the test suite, refer to the Lab 2 handout.

Note: When you first run `make` in `$LAB3_ROOT/build`, you'll encounter errors because the control unit in `riscvssc_part1` is incomplete. You'll need to make modifications before compiling and running tests. However, the checks can be performed on other designs.

3 Part 1: Implement Two-Wide Superscalar Processor (50%)

In this part, you are provided with a 7-stage, two-wide superscalar in-order processor, lacking the control logic, scoreboard, and instruction steering logic. Each stage of the processor can handle up to two instructions simultaneously.

The main objective of this part is to enable the processor to issue and execute two instructions simultaneously by implementing the instruction steering logic and scoreboard.

3.1 Processor Overview

The execute stage consists of two heterogeneous functional units:

- Pipeline A: Capable of executing all ALU, memory, control flow, and multiply/divide operations. It includes a four-stage multiply/divide unit.
- Pipeline B: Limited to simple ALU operations and cannot handle multiply, branch, jump, divide, or memory instructions.

Fetch Stage

The fetch stage should retrieve two instructions per cycle when not stalled. We provide a triple-ported memory unit, replacing the dual-ported one from the previous lab. This allows up to three memory accesses (two instructions and one data access) per cycle, preventing structural hazards.

Decode/Issue Stage

In lecture, we discussed the I4 and I2OI processor architectures, which separate the decode and issue stages. For continuity with the previous lab, however, we combine them into a single decode/issue stage in this lab.

The decode/issue stage should attempt to issue up to two instructions per cycle while respecting data dependencies and structural constraints.

Register File

To minimize structural hazards, we provide a six-ported register file, allowing four reads (two for each pipeline) and two writes per cycle.

Steering Logic

Since the two execute pipelines support different instruction types, instruction steering logic is required to assign decoded instructions to the appropriate pipeline.

Pipeline A can execute all supported instruction types, while Pipeline B can only execute simple ALU instructions.

Instruction 0 Type	Instruction 1 Type	Instruction 0 Pipeline	Instruction 1 Pipeline
ALU	ALU	A	B
ALU	Non-ALU	B	A
Non-ALU	ALU	A	B
Non-ALU	Non-ALU	A	Stall, then A

Table 1: Instruction Steering Rules

The steering logic should issue two instructions per cycle whenever possible. However, several constraints must be considered:

- If the second instruction has a RAW dependency on the first instruction, the two instructions cannot be issued simultaneously.
- If both instructions write to the same destination register, the second instruction must stall to avoid a WAW conflict.
- If both instructions require Pipeline A, they cannot be issued simultaneously and one instruction must be delayed.

Scoreboard

You will need to implement a scoreboard that is accessed during the decode/issue stage.

The scoreboard serves two main purposes:

- Determine whether the source registers of an instruction are ready.
- Determine the pipeline stage from which a pending value should be bypassed.

Dest. Reg.	Pending (Y/N)	Functional Unit (Pipeline A/B)	4	3	2	1	0
x0	0	0	0	0	0	0	0
x1	0	1	0	0	0	0	0
...	...	...	...	...	...	...	...

Table 2: Processor Scoreboard

Since all instructions write to the register file at the end of the pipeline, the scoreboard does not need to track writeback hazards. Instead, it is responsible for providing data-stall and bypass information.

Execute Stage

The execute stage is divided into two parallel pipelines.

Pipeline A is fully functional and is identical to the execute pipeline used in `riscvlong`. Pipeline B only supports simple ALU operations and does not contain memory or multiply/divide functionality.

Memory (X1) Stage

The X1 stage serves as the memory stage, similar to the previous labs. The provided triple-ported memory supports two instruction accesses and one data access in the same cycle.

X2/X3 Stages

The X2 and X3 stages are included primarily for timing and pipeline alignment. Although Pipeline B does not contain a multiply/divide unit, both pipelines contain these stages because the processor remains an in-order design.

Writeback Stage

The writeback stage operates similarly to the previous lab. The register file provides two write ports, allowing instructions from both pipelines to write back in the same cycle.

The control logic must nevertheless prevent two simultaneously issued instructions from writing to the same destination register.

3.2 Implementation Requirements

For this part, you must implement the instruction steering logic and scoreboard so that the processor can correctly issue up to two instructions per cycle.

Your design must correctly handle:

- RAW dependencies between simultaneously decoded instructions.
- WAW hazards between simultaneously decoded instructions.
- Instructions that can only execute on Pipeline A.
- Two non-ALU instructions decoded in the same cycle.
- Scoreboard-based source operand readiness checking.
- Operand bypassing from the appropriate pipeline stages.

For this part, you're only allowed to modify the following files:

- `riscvssc-CoreCtrl.v`
- `riscvssc-CoreScoreboard.v`

3.3 Functional Correctness

Functional correctness is the minimum requirement for this part.

Your processor must:

- Pass all provided RISC-V instruction tests under both normal and randomized-delay simulation.
- Pass all provided `ubmark` benchmarks under both normal and randomized-delay simulation.

An implementation that fails any required correctness test will not be considered functionally correct.

If any of the required tests fails, you will receive zero points for this part.

4 Part 2: Optimize Two-Wide Superscalar Processor (40%)

After completing the baseline two-wide superscalar processor in Part 1, your next task is to improve its performance. To begin, run the setup.sh in riscvssc_part2:

```
% cd $LAB3_ROOT/riscvssc_part2
% ./setup.sh
```

The setup.sh script copies files from /riscvssc_part1, renames any file names (and relevant build entries) replacing the riscvssc_part1 prefix with riscvssc_part2, and performs basic build updates.

Rebuild your project to verify every thing has been copied correctly:

```
% cd $LAB3_ROOT/build
% make
```

4.1 Optimization Objective

The objective of this part is to improve processor performance by making better use of the available hardware resources.

You should first analyze your Part 1 implementation and identify situations that cause unnecessary pipeline stalls or poor utilization of the two execution pipelines. Then, design and implement one or more optimizations to reduce these performance limitations.

Possible optimization directions include, but are not limited to:

- Improving instruction steering and pairing decisions.
- Reducing unnecessary stalls caused by one blocked instruction.
- Improving branch handling and speculative execution.
- Improving scoreboard or dependency-tracking mechanisms.
- Introducing a issue queue and reorder buffer mentioned in class to support more aggressive scheduling.

These are only examples. You are free to explore other architectural optimizations as long as they satisfy the resource constraints described below.

4.2 Resource Limitations

You may modify components beyond the control logic and scoreboard. However, all changes must follow the hardware constraints specified in this section. Performance improvements should come from better utilization of the available hardware rather than simply adding execution resources or reducing the pipeline depth.

Your optimized processor must satisfy the following resource constraints:

- The processor may use two ALU execution units. You may not instantiate additional ALUs to increase instruction throughput.
- The processor may use one multiply/divide functional unit. You may reorganize how instructions are scheduled to this unit, but you may not add additional multiply/divide units.
- Only two instruction-memory (`imem`) ports and one data-memory (`dmem`) port may be used. Therefore, at most two instructions may be fetched and one data-memory access may be issued in each cycle.
- The register file must remain a four-read, two-write register file. At most four source operands may be read from the register file and at most two destination registers may be written to the register file in a single cycle.

- The processor must retain the original 7-stage pipeline structure. Pipeline stages may not be merged or removed to reduce the total cycle count. Additional microarchitectural structures, such as issue queues and reorder buffers, are allowed to support more aggressive instruction scheduling as long as the original 7-stage pipeline structure is preserved and all resource constraints are satisfied.

Your design should focus on using the available resources more efficiently rather than obtaining higher performance by duplicating existing execution resources.

Any violation of the above rules will result in zero points for the entire lab.

4.3 Functional Correctness

Functional correctness is the minimum requirement for this part.

Your processor must:

- Pass all provided RISC-V instruction tests under both normal and randomized-delay simulation.
- Pass all provided `ubmark` benchmarks under both normal and randomized-delay simulation.

An implementation that fails any required correctness test will not be considered functionally correct.

If any of the required tests fails, you will receive zero points for this part.

4.4 Performance

In addition to functional correctness, your optimized processor must satisfy the cycle-count requirements shown below.

Three performance levels are provided:

- **Level 1:** The cycle limit must be satisfied for all listed benchmarks. (15%)
- **Level 2:** The cycle limit must be satisfied for all listed benchmarks. (5%)

Benchmark	Level 1 Maximum Cycles	Level 2 Maximum Cycles
`ubmark-vvadd`	1400	1200
`ubmark-bin-search`	1500	1300
`ubmark-cmplx-mult`	4000	3500
`ubmark-masked-filter`	8500	7500

Table 3: Cycle Requirements for Part 2

- **Level 3:** Linear ranking among implementations that meet Level 2. Performance is evaluated using two separate sets of benchmarks:
  - **Released Benchmarks (10%):** Performance are ranked based on the provided benchmark. The top-performing implementation receives 10 points, while the lowest-performing implementation receives 0 points.
  - **Hidden Benchmarks (10%):** Performance are ranked based on hidden benchmarks. The top-performing implementation receives 10 points, while the lowest-performing implementation receives 0 points.

5 Report (20%)

In addition to the source code, you must submit a lab report that clearly documents your implementation, design decisions, performance analysis, and evaluation results.

Part 1 (10%)

- **Design:** Describe the architecture and implementation of your Part 1 processor. Explain the major control logic, instruction steering, hazard handling, and any important design decisions.
- **Evaluation:** Report the simulation results and cycle counts for `riscvssc_part1`.
- **Grading Requirement:** Report score for part 1 is awarded only if the implementation is functionally correct. An implementation that fails the required correctness tests receives 0 points for this section.

Part 2 (10%)

- **Profiling:** Describe how you profiled the Part 1 design to identify its main performance bottlenecks. Explain which pipeline stages, hazards, stalls, or structural conflicts contribute the most overhead.
- **Optimization Strategy:** Based on your profiling results, explain which bottlenecks you chose to optimize and why.
- **Design:** Describe the microarchitectural changes made in Part 2. Explain the major mechanisms added or modified, such as scheduling, issue logic, bypassing, branch handling, hazard resolution, or other performance-oriented optimizations.
- **Evaluation:** Report the simulation results and cycle counts for `riscvssc_part2`. Compare the results against Part 1, quantify the performance improvement, and discuss which optimizations contribute most to the improvement.
- **Analysis:** Discuss any remaining performance bottlenecks or limitations in your Part 2 design. If applicable, describe possible improvements that could further increase performance.
- **Grading Requirement:** The Part 2 report is worth 10 points, with 5 points awarded for meeting the Level 1 cycle-count requirements and another 5 points for meeting the Level 2 requirements. Level 3 does not contribute additional report points, as it is evaluated through performance ranking. However, all optimizations implemented in Part 2, including those targeting Level 3, must be documented and analyzed in the report.

Submit your lab report as a single PDF file named `student_id-lab3-report.pdf`. The report must not exceed four pages, and hand-drawn figures are not allowed.

6 Submission

6.1 Deliverables

Submit a `.tar.gz` file of your working directory, keeping the original structure intact. All source files should be in `$LAB3_ROOT/riscvssc_part1` and `$LAB3_ROOT/riscvssc_part2`.

Be sure to clean up generated waveforms or compiled code by running:

```
% cd $LAB3_ROOT/build
% make clean
% cd $LAB3_ROOT/tests
% rm -rf build
% cd $LAB3_ROOT/ubmark
% rm -rf build
```

Create a tarball of your completed lab with:

```
% cd $LAB3_ROOT
% cd ..
% tar -cvzf student_id-lab3.tar.gz lab3
```

6.2 Submission Instructions

- Keep your code in the `lab3` folder. If the code is not in this tarball, we cannot grade it.
- Submit the tarball and lab report separately via e3:
  - `student_id-lab3-report.pdf`
  - `student_id-lab3.tar.gz`
- **Incorrect submission format:** A 10-point penalty will be applied if the required submission format is not followed.

7 Grading Rubric

- **Grading Note:** The total score for this lab is 110 points, consisting of 20 points for the report and 90 points for the code.
- **Report (20%)**
  - Part 1 (10%).
  - Part 2 (10%).
- **Code (90%)**
  - Part 1 (50%)
    - Functional correctness.
  - Part 2 (40%)
    - Level 1 performance (15%).
    - Level 2 performance (5%).
    - Performance ranking (20%).

8 Additional Notes

The rules for this lab are slightly different from those of the previous labs. If you have any questions regarding the grading criteria or implementation requirements, feel free to post them on the forum.